Министерство образования Республики Беларусь

Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Информационные сети. Основы безопасности»

ОТЧЁТ
к лабораторной работе №1
на тему
«ОСНОВЫ ИНФОРМАЦИОННЫХ СЕТЕЙ»
БГУИР 6-05-0612-02 23

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ГЕРЧИК Артем Вадимович

_____
(дата, подпись преподавателя)

Минск 2026

1 ПОСТАНОВКА ЗАДАЧИ

Целью работы является проектирование и развертывание локальной сети с использованием статической IP-адресации, обеспечивающей фиксированное назначение сетевых параметров узлам. В рамках задания необходимо определить адресное пространство и назначить фиксированные IP-адреса узлам сети. Конфигурация должна быть воспроизводимой и однозначно интерпретируемой при последующей проверке.

Верификация конфигурации включает проверку адресов интерфейсов, таблиц маршрутизации и возможности передачи пакетов между устройствами с использованием диагностических команд. Результаты проверок должны подтвердить корректность назначенных IP-адресов, правильность конфигурации подсети, а также доступность узлов на сетевом и прикладном уровнях.

2 СОЗДАНИЕ СЕТИ

В работе использована контейнерная эмуляция сети в Docker [1]. Была создана сеть *lab1_labnet* с адресным пространством 10.10.5.0/24, а контейнерам *h1* и *h2* через docker-compose назначены статические адреса 10.10.5.10 и 10.10.5.11 соответственно. Это обеспечило воспроизводимую адресацию и позволило однозначно интерпретировать результаты проверок сетевой достижимости.

В образ контейнеров добавлены необходимые сетевые утилиты для диагностики и тестирования; в данной работе использовались ping, netcat и tcpdump. Контейнеры запускались командой docker-compose up -d, а корректность конфигурации проверялась просмотром интерфейсов и таблиц маршрутизации внутри контейнеров с помощью ip addr и ip route. Результат этих команд приведен на рисунке 2.1.

```
velebes@arch ~/Study/ISOB/LR1/Lab1> docker exec h1 ip addr show dev eth0
2: eth0@if20: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 62:30:27:c2:84:eb brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.10.5.10/24 brd 10.10.5.255 scope global eth0
       valid_lft forever preferred_lft forever
velebes@arch ~/Study/ISOB/LR1/Lab1> docker exec h2 ip addr show dev eth0
2: eth0@if19: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 7a:c1:fd:78:da:ec brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.10.5.11/24 brd 10.10.5.255 scope global eth0
       valid_lft forever preferred_lft forever
```

Рисунок 2.1 – Вывод ip addr show для h1 и h2, демонстрирующий адреса хостов

Проверка сетевой достижимости началась с ICMP-обмена. Пинг между хостами показал достижимость адресов и стабильное время отклика, что свидетельствует о корректной работе L2/L3-настроек и маршрутизации в пределах сети. На рисунке 2.2 показаны вывод ping и параллельный захват трафика tcpdump, подтверждающий прохождение ICMP-запросов и ответов [2].

```
velebes@arch ~/Study/ISOB/LR1/Lab1> docker exec h1 ping -c 8 10.10.5.11
PING 10.10.5.11 (10.10.5.11) 56(84) bytes of data.
64 bytes from 10.10.5.11: icmp_seq=1 ttl=64 time=0.050 ms
64 bytes from 10.10.5.11: icmp_seq=2 ttl=64 time=0.053 ms
64 bytes from 10.10.5.11: icmp_seq=3 ttl=64 time=0.049 ms
64 bytes from 10.10.5.11: icmp_seq=4 ttl=64 time=0.060 ms
64 bytes from 10.10.5.11: icmp_seq=5 ttl=64 time=0.044 ms
64 bytes from 10.10.5.11: icmp_seq=6 ttl=64 time=0.060 ms
64 bytes from 10.10.5.11: icmp_seq=7 ttl=64 time=0.056 ms
64 bytes from 10.10.5.11: icmp_seq=8 ttl=64 time=0.049 ms

--- 10.10.5.11 ping statistics ---
8 packets transmitted, 8 received, 0% packet loss, time 7147ms
rtt min/avg/max/mdev = 0.044/0.052/0.060/0.005 ms
velebes@arch ~/Study/ISOB/LR1/Lab1>
```

```
velebes@arch ~/Study/ISOB/LR1/Lab1> docker exec -it h2 tcpdump -n -i any icmp -c 16
tcpdump: data link type LINUX_SLL2
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
13:07:22.826712 eth0  In  IP 10.10.5.10 > 10.10.5.11: ICMP echo request, id 14360, seq 1, length 64
13:07:22.826727 eth0  Out IP 10.10.5.11 > 10.10.5.10: ICMP echo reply, id 14360, seq 1, length 64
13:07:23.829706 eth0  In  IP 10.10.5.10 > 10.10.5.11: ICMP echo request, id 14360, seq 2, length 64
13:07:23.829722 eth0  Out IP 10.10.5.11 > 10.10.5.10: ICMP echo reply, id 14360, seq 2, length 64
13:07:24.853684 eth0  In  IP 10.10.5.10 > 10.10.5.11: ICMP echo request, id 14360, seq 3, length 64
13:07:24.853701 eth0  Out IP 10.10.5.11 > 10.10.5.10: ICMP echo reply, id 14360, seq 3, length 64
13:07:25.877694 eth0  In  IP 10.10.5.10 > 10.10.5.11: ICMP echo request, id 14360, seq 4, length 64
13:07:25.877718 eth0  Out IP 10.10.5.11 > 10.10.5.10: ICMP echo reply, id 14360, seq 4, length 64
13:07:26.901660 eth0  In  IP 10.10.5.10 > 10.10.5.11: ICMP echo request, id 14360, seq 5, length 64
13:07:26.901677 eth0  Out IP 10.10.5.11 > 10.10.5.10: ICMP echo reply, id 14360, seq 5, length 64
13:07:27.925714 eth0  In  IP 10.10.5.10 > 10.10.5.11: ICMP echo request, id 14360, seq 6, length 64
13:07:27.925733 eth0  Out IP 10.10.5.11 > 10.10.5.10: ICMP echo reply, id 14360, seq 6, length 64
13:07:28.950703 eth0  In  IP 10.10.5.10 > 10.10.5.11: ICMP echo request, id 14360, seq 7, length 64
13:07:28.950721 eth0  Out IP 10.10.5.11 > 10.10.5.10: ICMP echo reply, id 14360, seq 7, length 64
13:07:29.973689 eth0  In  IP 10.10.5.10 > 10.10.5.11: ICMP echo request, id 14360, seq 8, length 64
13:07:29.973706 eth0  Out IP 10.10.5.11 > 10.10.5.10: ICMP echo reply, id 14360, seq 8, length 64
```

Рисунок 2.2 – Вывод ping и захват tcpdump, подтверждающие ICMP-обмен

После подтверждения базовой достижимости выполнена передача данных по TCP с использованием *netcat* [3]. Передача данных осуществлялась в интерактивном режиме между хостами. На узле h2 был запущен слушатель командой *nc -l -p 12345*, а на узле h1 выполнена отправка данных командой *nc 10.10.5.11 12345*. Это установило интерактивную сессию, в которой введённый текст появлялся в реальном времени на принимающей стороне, что подтверждает корректность работы прикладного уровня. На рисунке 2.3 приведён фрагмент интерактивной передачи по TCP.

```
velebes@arch ~/Study/ISOB/LR1/Lab1> docker exec -it h1 sh -c "nc 10.10.5.11 12345"
test
hello from h1
test tsets
ayoo
```

```
velebes@arch ~/Study/ISOB/LR1/Lab1> docker exec -it h2 sh -c "nc -l -p 12345"
test
hello from h1
test tsets
ayoo
```

Рисунок 2.3 – Передача данных по TCP с использованием netcat

ЗАКЛЮЧЕНИЕ

Реализация локальной сети со статической IPv4-адресацией была выполнена на базе технологии контейнеризации Docker. Применение контейнерной виртуализации обеспечило изолированную и воспроизводимую среду с фиксированными сетевыми параметрами. Верификация конфигурации проводилась на двух уровнях: ICMP-запросы продемонстрировали базовую достижимость узлов на сетевом уровне, а интерактивное TCP-соединение через netcat подтвердило корректность работы всего стека протоколов, включая транспортный и прикладной уровни.

Существенным преимуществом выбранного подхода является использование эмуляции, при которой контейнеры Docker задействуют реальный сетевой стек ядра Linux с полноценной реализацией протоколов TCP/IP. Это обеспечивает максимально реалистичное представление работы сетевых протоколов, в отличие от симуляторов с упрощёнными моделями.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Networking overview [Электронный ресурс]. – Режим доступа: https://docs.docker.com/network/. – Дата доступа: 07.02.2026.

[2] How to use tcpdump command on Linux [Электронный ресурс]. – Режим доступа: https://linuxconfig.org/how-to-use-tcpdump-command-on-linux. – Дата доступа: 07.02.2026.

[3] nc(1) - Linux manual page [Электронный ресурс]. – Режим доступа: https://linux.die.net/man/1/nc. – Дата доступа: 07.02.2026.